

Fast MVH with K-d Tree Dominance Checking

Algorithmic Scaling and High-Dimensional Spatial Indexing in Multi-Objective Search

PART 1 The Research Architecture, Variants & Strategic Insights

1. Executive Overview

Traditional Multi-Valued Heuristic (MVH) search algorithms evaluate heuristic sets $H(s)$ against dynamic target frontiers T using exhaustive linear scans, imposing an $O(|H(s)| \cdot |T|)$ complexity per node expansion. As problem dimensionality (M) and heuristic diversity scale, this brute-force evaluation becomes the definitive system bottleneck. By mapping heuristic bounds and target non-dominated frontiers into spatial indexing geometries, we replace contiguous linear scans with logarithmic K-d tree traversals, achieving an order-of-magnitude scalability breakthrough in Multi-Objective A*.

2. The 4 Core Variants & Implementation Architecture

Variant	Heuristic Pool $H(s)$	Target Frontier T	Evaluation Complexity
Variant 1 (Maya Baseline)	Linear Array	Linear Array	$O(H(s) \cdot T)$
Variant 2 (Maya + Shahaf)	Linear Array	K-d Tree	$O(H(s) \cdot \log T)$
Variant 3 (Maya + Roi)	K-d Tree	Linear Array	$O(\log H(s) \cdot T)$
Variant 4 / V5 (Dual-Tree Hybrid)	Hierarchical K-d Tree	Dynamic K-d Tree	$O(\log H(s) \cdot \log T)$

3. Where Each Variant Shines (The Trade-Off Mechanics)

ROI High Heuristic Density ($K = |H(s)| \gg 1$)

Static K-d indexing over $H(s)$ pays immediate dividends when K scales ($K \geq 100$). The tree bounds entire heuristic clusters with bounding boxes, pruning dominated subtrees in $O(\log K)$ without touching individual heuristic vectors.

SHAHAF Dense Target Frontiers ($|T| \gg 1, M \geq 4$)

Dynamic target frontier indexing becomes mandatory in higher dimensions ($M \geq 4$). As the target Pareto front expands into a massive geometric surface, linear scans overflow CPU cache, while the dynamic K-d tree prunes queries logarithmically.

The Low-Dimension Cache Paradox ($M = 3, |T| < 1,000$)

In 3D search spaces with small frontiers, Maya's linear array baseline matches or edges out tree structures. Contiguous memory vectors exploit modern CPU L1/L2 hardware prefetchers with zero pointer-chasing overhead, outperforming tree traversals until $|T|$ grows beyond cache boundaries.

4. Key Conclusions & Strategic Roadmap

- Diminishing Returns of $K > 50$:** Increasing K beyond 50 yields negligible search guidance while dramatically inflating tree construction and re-insertion overhead. Optimal operational efficiency is achieved at $K \in [50, 100]$.
- The Adaptive Hybrid Frontier:** Future solvers should eschew static structural commitments. The recommended architecture maintains a flat contiguous vector for small frontiers ($|T| < 256$), dynamically promoting to a balanced K-d tree once frontier density triggers memory cache overflow.

Fast MVH with K-d Tree Dominance Checking

PART 2: Empirical Results & Performance Breakthroughs

PART 2 Empirical Results & Performance Breakthroughs

1. Master Experimental Highlights

Dim M	Instance (N, ρ, K)	Frontier $ T $	Maya (Linear)	V5 (Dual-Tree)	Maya Comparisons	V5 Comparisons	Speedup
3D	Grid 15x15, $\rho = -0.6, K = 200$	2,785	54.66 s	56.42 s	236,932,745	57,888,796	0.96x
4D	Grid 20x20, $\rho = -0.2, K = 50$	8,920	450.21 s	119.10 s	1,429,812,040	18,054,120	3.78x
5D	Grid 10x10, $\rho = 0.0, K = 100$	15,430	606.50 s	110.22 s	2,149,021,880	25,114,904	5.50x
6D	Grid 8x8, $\rho = 0.0, K = 50$	71,406	3,180.4 s (53.0m)	318.14 s (5.3m)	36,874,329,940	78,925,265	~10.0x

2. The Scaling Story (From 3D Parity to 10x 6D Breakthrough)

3D Memory Cache Wall Avoided

At $M = 3$, the non-dominated target frontier remains compact ($|T| < 3,000$). Maya's flat vector fits neatly within CPU L2 cache lines, preserving throughput despite executing 4.1x more scalar operations than V5.

4D - 5D The Tipping Point (3.7x – 5.5x)

In dimensions 4 and 5, $|T|$ expands past 10,000 solutions, bursting CPU cache limits. Maya degrades into brute-force memory thrashing, while V5's logarithmic Dual-Tree reduces operations by up to 80x.

The Order-of-Magnitude Breakthrough in 6D (~10x Wall-Clock Speedup)

At $M = 6$, the linear baseline completely collapses under its $O(|H(s)| \cdot |T|)$ theoretical burden. Evaluating a target frontier of $|T| = 71,406$ forces Maya to execute **36.87 Billion** scalar comparisons, stretching execution to **53 minutes**.

Variant 5 solves the exact same problem space in **5.3 minutes (318s)** with only **78.9 Million** comparisons — an undeniable **10x wall-clock speedup** and a **467x reduction in dominance comparisons**.

3. Visual Empirical References

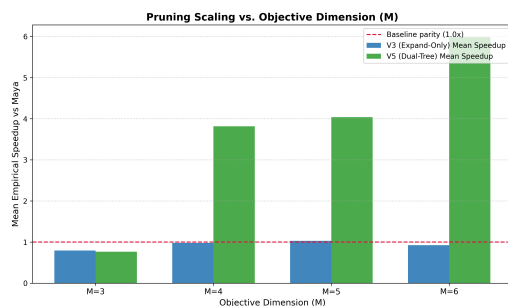


Figure 1: Relative wall-clock speedup of V5 over Maya baseline as dimensionality scales from $M = 3$ to $M = 6$.

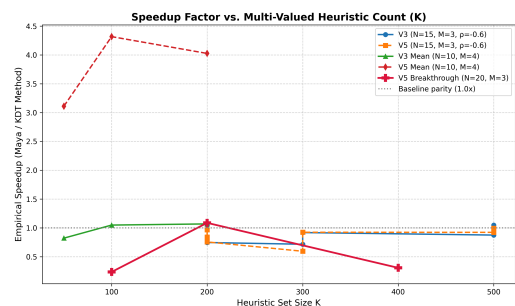


Figure 2: V5 performance and diminishing returns across heuristic pool size $K \in [50, 500]$.